

ABSTRACT

The project, "**IoT-Based Energy Consumption and Usage Analysis**," focuses on designing a system that uses IoT technologies to monitor, analyse, and optimize energy usage in real-time. The system is built around the **NodeMCU ESP32**, which interfaces with **ZMPT101B voltage sensors** and **ACS712 current sensors** to collect accurate data from connected appliances. These sensors measure voltage, current, and power consumption, enabling the system to calculate energy usage and cost estimations effectively.

The data collected from the sensors is processed and stored in structured formats such as **Excel** and **CSV** files. These datasets are then imported into **Power BI** for visualization, offering insights like energy usage trends, cost breakdowns, and appliance-specific consumption patterns. These visualizations help users optimize energy consumption and reduce costs.

The system integrates real-time monitoring, historical data analysis, and actionable insights to enhance energy efficiency. The use of reliable sensors ensures high accuracy in the recorded measurements, making the system suitable for everyday applications.

1.INTRODUCTION

The integration of IoT into everyday applications has revolutionized how we interact with technology, especially in monitoring and automation. This mini project, "**IoT-Based Energy Consumption and Usage Analysis**," focuses on designing a system to efficiently monitor and analyze energy usage through IoT technologies. By combining advanced microcontroller technology with precise sensor integration, this project offers a robust solution for real-time energy management and optimization.

At the heart of the system is the **NodeMCU ESP32**, a versatile microcontroller with built-in Wi-Fi, ideal for IoT applications. It is paired with **ZMPT101B voltage sensors** and **ACS712 current sensors** for accurate measurement of voltage, current, and power consumption.

The project's primary goal is to explore key aspects of IoT-based energy monitoring, such as sensor integration, microcontroller programming, and data analysis. This hands-on experience not only enhances our understanding of energy systems but also provides valuable insights into automation and sustainable practices.

This report covers the conceptualization, design, and implementation of the IoT-based energy monitoring system. It delves into the hardware and software components, discusses challenges faced during implementation, and presents the results from experimental testing. Through this project, we aim to contribute to the growing field of IoT-based energy management while fulfilling academic and practical objectives, ultimately promoting efficient energy usage and deepening the understanding of IoT applications in everyday life.

2.SYSTEM ANALYSIS

2.1 IMPLEMENTED SYSTEM

2.1.1 PROPOSED SYSTEM

The IoT-Based Energy Consumption and Usage Analysis system is designed to monitor and analyze energy consumption in real-time through the integration of IoT technologies. The system aims to provide accurate data on energy usage by measuring key parameters such as voltage, current, and power consumption of connected devices. By using the NodeMCU ESP32 microcontroller, the system transmits the sensor data to Excel files for storage and further analysis. The data is then processed and visualized using Power BI to provide actionable insights.

The use of ZMPT101B voltage sensors and ACS712 current sensors ensures precise data collection, enabling users to monitor their energy consumption remotely. This system aims to enhance energy efficiency by providing insights into consumption patterns, helping users optimize their energy usage and reduce costs.

2.1.2 FEATURES

- **Real-time Energy Monitoring:**

The system measures and displays real-time energy consumption data, including voltage, current, and power usage.

- **Local Data Storage in Excel:**

Energy consumption data is stored locally in an Excel file for analysis and easy reference, eliminating the need for cloud storage.

- **Accurate Measurement:**
Uses ZMPT101B voltage sensors and ACS712 current sensors to ensure precise energy usage data collection.
- **Cost Estimation:**
Calculates the estimated cost of energy consumption based on the recorded data, helping users understand their energy expenses.
- **Basic Energy Insights:**
Provides simple insights into energy usage, helping users identify patterns and potential areas for efficiency improvements.

2.1.3 FUNCTIONALITY

A. DATA COLLECTION AND STORAGE:

The system continuously monitors the energy consumption of connected appliances using the ACS712 current sensor and ZMPT101B voltage sensor. As the data is collected, it is processed by the Arduino Uno and saved into a time-stamped format in CSV files. These CSV files store real-time information such as current, voltage, and power consumption for each appliance. The data is organized and saved in a structured folder, ready to be used for further analysis.

B. DATA STREAMING:

The energy usage data is recorded continuously, ensuring that the system collects up-to-date information. The CSV files are stored in sequence where each file represents a new dataset captured at a specific time. This structured data storage enables easy tracking of historical energy usage trends.

C. INTEGRATION WITH POWER BI:

Once the data is stored in the folder it can be imported into Power BI for data visualization. Power BI uses the CSV files as a data source, enabling you to create dynamic reports and dashboards that visualize trends such as energy consumption over time, voltage fluctuations, and current usage patterns.

D. REAL-TIME DATA VISUALIZATION:

The data stored in the csv files can be updated regularly to reflect real-time changes in energy usage. Power bi then processes the latest data, displaying it on interactive dashboards where users can view energy consumption in various formats, such as line charts, bar graphs, and tables. This helps in making data-driven decisions on energy optimization.

E. DATA ANALYSIS AND INSIGHTS:

The collected data is not just stored but also analysed using Power BI's built-in features. This allows for insights into energy consumption patterns, identifying high-energy usage periods, and detecting anomalies in power consumption. Users can set alerts or thresholds within Power BI to notify them of significant changes or excessive energy usage.

2.1.4 ADVANTAGES

- **Energy Efficiency Optimization:**
Helps users monitor and reduce energy consumption through actionable data.
- **Cost-effective Solution:**
Avoids cloud integration costs by storing data locally in Excel, making it a budget-friendly system.
- **User-Friendly Implementation:**
Designed with simplicity, making it accessible to users with minimal technical knowledge.

- **Accurate and Reliable:**

Uses calibrated sensors for dependable energy data, ensuring trustworthy results.

- **Supports Financial Planning:**

The cost estimation feature enables users to manage and budget energy expenses effectively.

2.2 SYSTEM SPECIFICATION

2.2.1 HARDWARE REQUIREMENTS

A. NODEMCU ESP32 MICROCONTROLLER:



Fig.2.2.1.A NODEMCU ESP32

The NodeMCU ESP32 microcontroller serves as the central processing unit for the IoT-based energy consumption monitoring system. It facilitates the collection of data from sensors, processes it, and communicates with the cloud or other devices. Its versatility, integrated Wi-Fi and Bluetooth capabilities, and ease of programming make it an ideal choice for this energy monitoring project.

MICROCONTROLLER:

- ESP32 (32-bit dual-core processor)

OPERATING VOLTAGE:

- 3.3V

INPUT VOLTAGE (RECOMMENDED):

- 5V via Micro USB

INPUT VOLTAGE (LIMITS):

- 3.3V to 5V

DIGITAL I/O PINS:

- 34 (including PWM, ADC, and GPIO pins)

ANALOG INPUT PINS:

- 12 (for reading sensor data such as voltage and current)

DC CURRENT PER I/O PIN:

- 12 mA (max)

FLASH MEMORY:

- 4 MB (used for program storage)

SRAM:

- 520 KB (used for data processing)

EEPROM:

- Not available (use SPIFFS or flash for storage needs)

CLOCK SPEED:

- 240 MHz (dual-core processor)

USB CONNECTOR:

- Micro USB (for programming and power)

POWER JACK:

- 2.1mm barrel jack for power input, typically 5V

VOLTAGE REGULATOR:

- Onboard regulator to handle input from 5V to 3.3V

RESET BUTTON:

- Used to restart the program or reset the ESP32 microcontroller.

PINOUT DIAGRAM

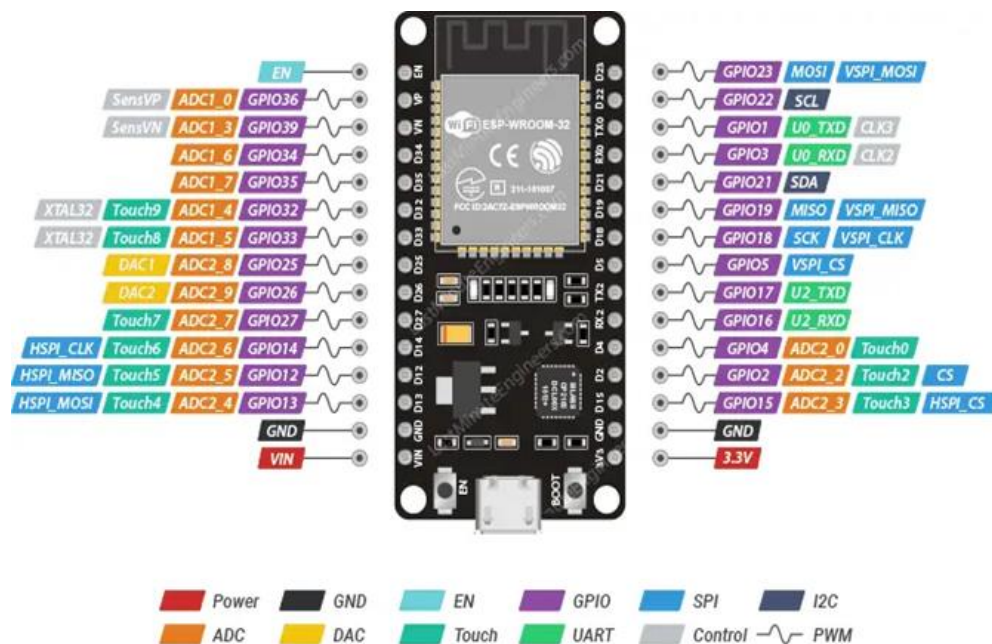


Fig.2.2.1.A PINOUT DIAGRAM

B. CURRENT SENSOR (ACS712)

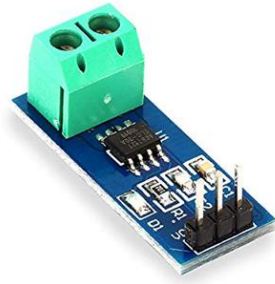


Fig.2.2.1.B CURRENT SENSOR

Description: The ACS712 current sensor is used to measure the current flowing through appliances, providing an accurate reading of the power consumption.

Purpose: This sensor is crucial for monitoring the energy usage of connected appliances. It provides real-time data that helps track the current consumption of devices connected to the system, enabling users to identify high-energy appliances and optimize usage.

C. VOLTAGE SENSOR (ZMPT101B)



Fig.2.2.1.C Voltage sensor

- **Description:** The ZMPT101B is a voltage sensor designed to measure the voltage supplied to appliances. It ensures accurate tracking of voltage levels in the circuit.
- **Purpose:** This sensor plays a vital role in calculating the power consumption of appliances. By measuring the voltage and combining it with current data, it allows precise power usage calculations ($P = V \times I$), making the energy consumption data reliable.

E. BULB (SAMPLE APPLIANCE)



Fig.2.2.1.E Bulb

- **Description:** A standard household bulb is used in this project as a representative appliance to simulate energy usage.
- **Purpose:** The bulb serves as a test load in the system, helping demonstrate how the energy consumption data is monitored and displayed. It is a simple appliance to test and showcase the system's functionality before connecting to more complex devices.

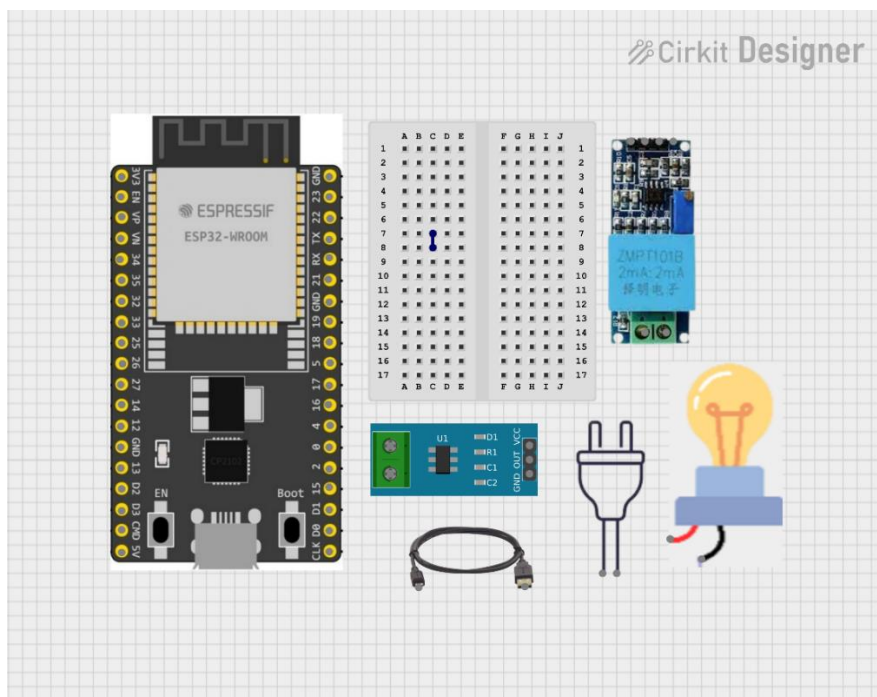


Fig.2.2.1.F COMPONENTS

2.2.2 SOFTWARE REQUIREMENTS

- **IDE:**
 - **Arduino IDE (Version 1.8.19):** Used for programming the Arduino Uno to read sensor data and control the system.
- **Data Storage:**
 - **Excel (Version 2021):** Stores the recorded energy data in CSV format for analysis.
- **Visualization:**
 - **Power BI (Version 2.114.1376.0):** Visualizes the energy data from the CSV files, helping users analyze energy consumption trends.
- **Libraries:**
 - **Arduino Libraries:** Libraries to help interface with sensors (ACS712, ZMPT101B) and store data.

3. SYSTEM DESIGN

3.1 METHODOLOGY

3.1.1 CIRCUIT DIAGRAM

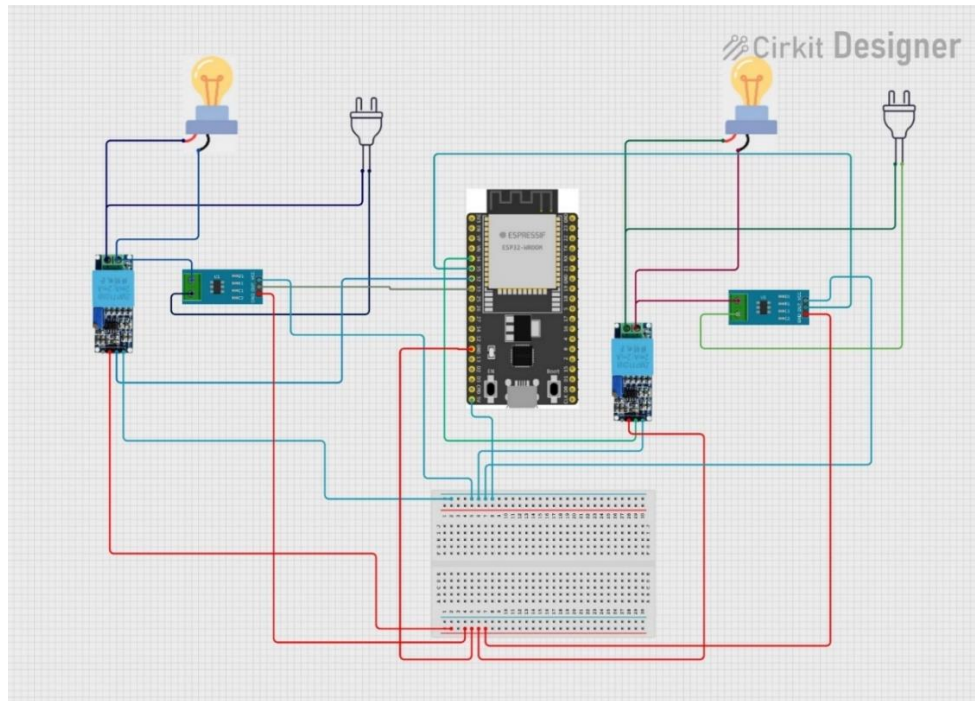


Fig.3.1.1 Circuit diagram

3.1.2 BLOCK DIAGRAM

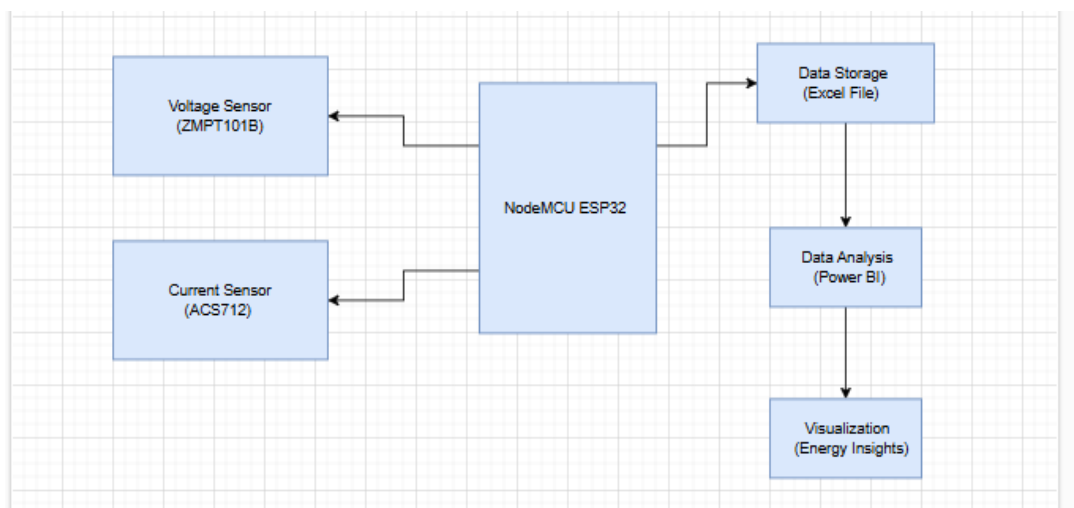


Fig.3.1.2 Block diagram

3.1.3 FLOW CHART

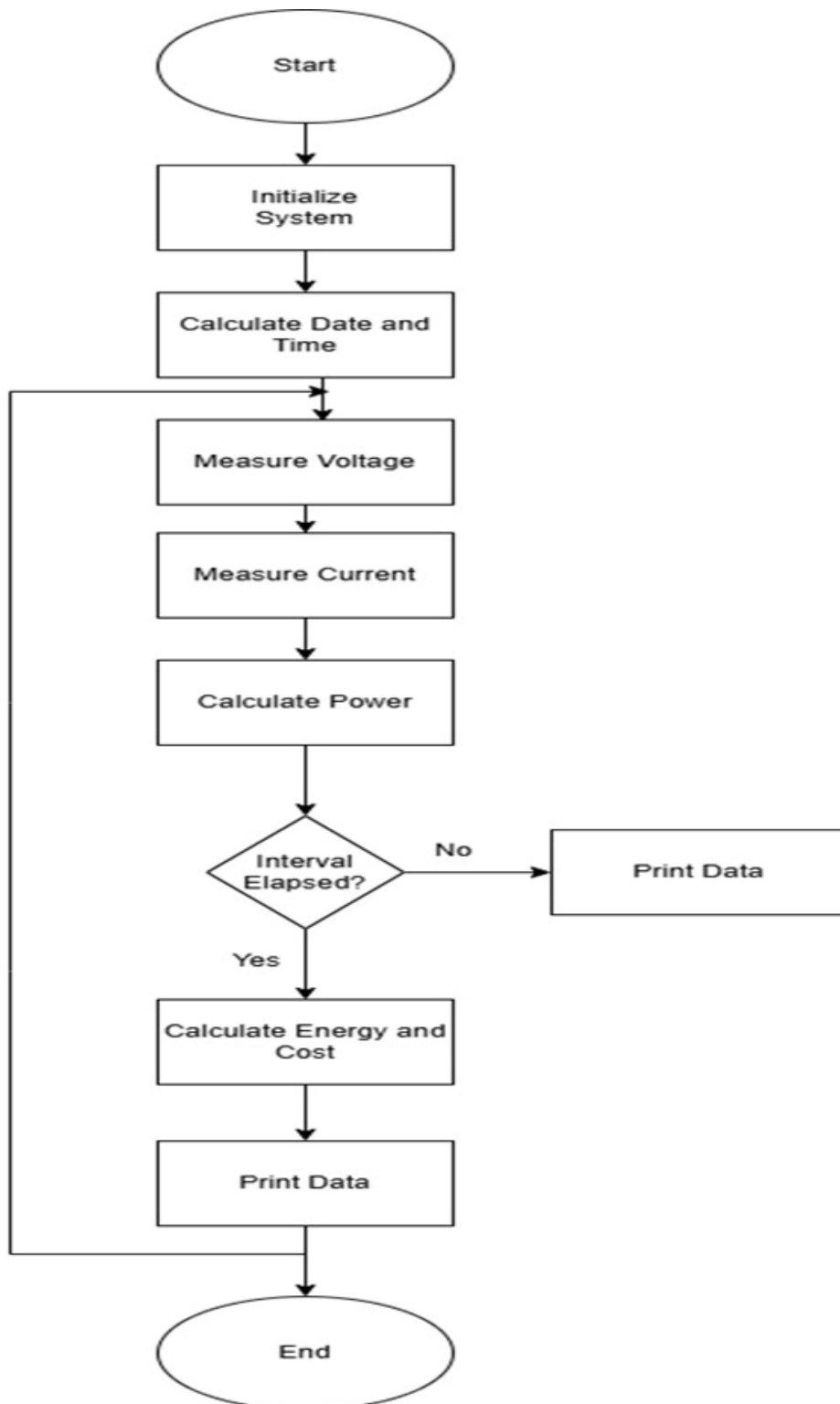


Fig.3.1.3 Flow Chart

3.2 WORKING PRINCIPLES

1. Current and Voltage Sensing:

Current Sensor (ACS712): Measures the current flowing through appliances and sends real-time data to the Arduino Uno.

Voltage Sensor (ZMPT101B): Measures the voltage supplied to appliances, essential for calculating power consumption ($P = V \times I$).

2. Data Collection:

The Arduino Uno processes data from the sensors and calculates power consumption in real time. Data is stored periodically (e.g., every minute or hour).

3. Data Storage:

Data is saved in CSV files (e.g., D1.csv, D2.csv) in a designated folder. Each file contains time-stamped data on current, voltage, and power values for historical tracking.

4. Data Visualization:

Power BI visualizes the stored data, displaying real-time and historical energy consumption in charts, graphs, and tables. Metrics like total consumption, cost estimation, and energy-saving trends are shown for optimization.

5. User Interface:

Users can monitor appliance status, view energy trends, and receive alerts for abnormal consumption or performance. Real-time data on power consumption, voltage, and current is displayed for each appliance.

4. IMPLEMENTATION

4.1 SOURCE CODE

```
#include <ZMPT101B.h>

#define SENSITIVITY_60W 495.0f
#define SENSITIVITY_100W 315.0f
#define MOVING_AVG_SIZE 10

// 60W Bulb Setup

ZMPT101B voltageSensor60W(34, 50.0);

float voltageReadings60W[MOVING_AVG_SIZE];

int voltageIndex60W = 0;

const int sensorPin60W = 35;

int mVperAmp60W = 315;

double currentReadings60W[MOVING_AVG_SIZE];

int currentIndex60W = 0;

double totalCurrent60W = 0;

double averageCurrent60W = 0;

double Voltage60W = 0;

double VRMS60W = 0;

double AmpsRMS60W = 0;

double Power60W = 0;

double Energy60W = 0;

double Cost60W = 0;
```

```

// 100W Bulb Setup

ZMPT101B voltageSensor100W(32, 50.0);

float voltageReadings100W[MOVING_AVG_SIZE];

int voltageIndex100W = 0;

const int sensorPin100W = 33;

int mVperAmp100W = 245;

double currentReadings100W[MOVING_AVG_SIZE];

int currentIndex100W = 0;

double totalCurrent100W = 0;

double averageCurrent100W = 0;

double Voltage100W = 0;

double VRMS100W = 0;

double AmpsRMS100W = 0;

double Power100W = 0;

double Energy100W = 0;

double Cost100W = 0;

unsigned long previousMillis = 0;

unsigned long interval = 20000;

unsigned long programStartTime = millis();

double costPerUnit = 4.80;

// Starting date and time (year, month, day, hour, minute, second)

int year = 2024;

int month = 11;

```

```

int day = 16;

int hour = 12;

int minute = 0;

int second = 0;

void setup() {

    Serial.begin(115200);

    voltageSensor60W.setSensitivity(SENSITIVITY_60W);

    voltageSensor100W.setSensitivity(SENSITIVITY_100W);

    Serial.println("Date,Time,Voltage (V),Current (Amps),Power (Watts),Energy (Wh),Cost
(Rs),Voltage (100W),Current (100W),Power (100W),Energy (100W),Cost (100W)");

}

void loop() {

    // Update date and time based on elapsed time

    updateDateTime();

    // Measure voltage and current for 60W bulb

    float voltage60W = voltageSensor60W.getRmsVoltage();

    voltageReadings60W[voltageIndex60W] = voltage60W;

    voltageIndex60W = (voltageIndex60W + 1) % MOVING_AVG_SIZE;

    float averageVoltage60W = 0;

    for (int i = 0; i < MOVING_AVG_SIZE; i++) {

        averageVoltage60W += voltageReadings60W[i];

    }

    averageVoltage60W /= MOVING_AVG_SIZE;

```

```

Voltage60W = getVPP(sensorPin60W);

VRMS60W = (Voltage60W / 2.0) * 0.707;

AmpsRMS60W = (VRMS60W * 1000) / mVperAmp60W;

totalCurrent60W -= currentReadings60W[currentIndex60W];

currentReadings60W[currentIndex60W] = AmpsRMS60W;

totalCurrent60W += currentReadings60W[currentIndex60W];

currentIndex60W = (currentIndex60W + 1) % MOVING_AVG_SIZE;

averageCurrent60W = totalCurrent60W / MOVING_AVG_SIZE;

Power60W = averageVoltage60W * averageCurrent60W;

// Measure voltage and current for 100W bulb

float voltage100W = voltageSensor100W.getRmsVoltage();

voltageReadings100W[voltageIndex100W] = voltage100W;

voltageIndex100W = (voltageIndex100W + 1) % MOVING_AVG_SIZE;

float averageVoltage100W = 0;

for (int i = 0; i < MOVING_AVG_SIZE; i++) {

    averageVoltage100W += voltageReadings100W[i];

}

averageVoltage100W /= MOVING_AVG_SIZE;

Voltage100W = getVPP(sensorPin100W);

VRMS100W = (Voltage100W / 2.0) * 0.707;

AmpsRMS100W = (VRMS100W * 1000) / mVperAmp100W;

totalCurrent100W -= currentReadings100W[currentIndex100W];

currentReadings100W[currentIndex100W] = AmpsRMS100W;

```

```

totalCurrent100W += currentReadings100W[currentIndex100W];

currentIndex100W = (currentIndex100W + 1) % MOVING_AVG_SIZE;

averageCurrent100W = totalCurrent100W / MOVING_AVG_SIZE;

Power100W = averageVoltage100W * averageCurrent100W;

unsigned long currentMillis = millis();

if (currentMillis - previousMillis >= interval) {

    previousMillis = currentMillis;

    Energy60W += Power60W * (interval / 3600000.0);

    Energy100W += Power100W * (interval / 3600000.0);

    double energyInKWh60W = Energy60W / 1000.0;

    double energyInKWh100W = Energy100W / 1000.0;

    Cost60W = energyInKWh60W * costPerUnit;

    Cost100W = energyInKWh100W * costPerUnit;

}

// Print results with separated columns

Serial.print(year);

Serial.print("-");

Serial.print(month);

Serial.print("-");

Serial.print(day);

Serial.print(",");

Serial.print(hour);

Serial.print(":");

```

```
Serial.print(minute);

Serial.print(":");

Serial.print(second);

Serial.print(",");

Serial.print(averageVoltage60W);

Serial.print(",");

Serial.print(averageCurrent60W);

Serial.print(",");

Serial.print(Power60W);

Serial.print(",");

Serial.print(Energy60W);

Serial.print(",");

Serial.print(Cost60W);

Serial.print(",");

Serial.print(averageVoltage100W);

Serial.print(",");

Serial.print(averageCurrent100W);

Serial.print(",");

Serial.print(Power100W);

Serial.print(",");

Serial.print(Energy100W);

Serial.print(",");

Serial.println(Cost100W);
```

```

    delay(1000);

}

// Function to update date and time based on millis()

void updateDateTime() {

    unsigned long elapsedMillis = millis() - programStartTime;

    second = elapsedMillis / 1000;

    if (second >= 60) {

        minute += second / 60;

        second %= 60;

    }

    if (minute >= 60) {

        hour += minute / 60;

        minute %= 60;

    }

    if (hour >= 24) {

        day += hour / 24;

        hour %= 24;

    }

    // Adjust days for the month

    int daysInMonth = getDaysInMonth(year, month);

    if (day > daysInMonth) {

        day -= daysInMonth;

        month++;
    }
}

```

```

if (month > 12) {

    month = 1;

    year++;

}

}

}

// Function to return the number of days in a month

int getDaysInMonth(int year, int month) {

    if (month == 2) {

        // Check for leap year

        if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {

            return 29;

        } else {

            return 28;

        }

    } else if (month == 4 || month == 6 || month == 9 || month == 11) {

        return 30;

    } else {

        return 31;

    }

}

// Function to calculate peak-to-peak voltage

float getVPP(int sensorPin) {

```

```
float result;

int readValue;

int maxValue = 0;

int minValue = 4096;

uint32_t start_time = millis();

while ((millis() - start_time) < 2000) {

    readValue = analogRead(sensorPin);

    if (readValue > maxValue) maxValue = readValue;

    if (readValue < minValue) minValue = readValue;

}

result = ((maxValue - minValue) * 3.3) / 4096.0;

return result;

}
```

5.EXPERIMENTAL RESULT

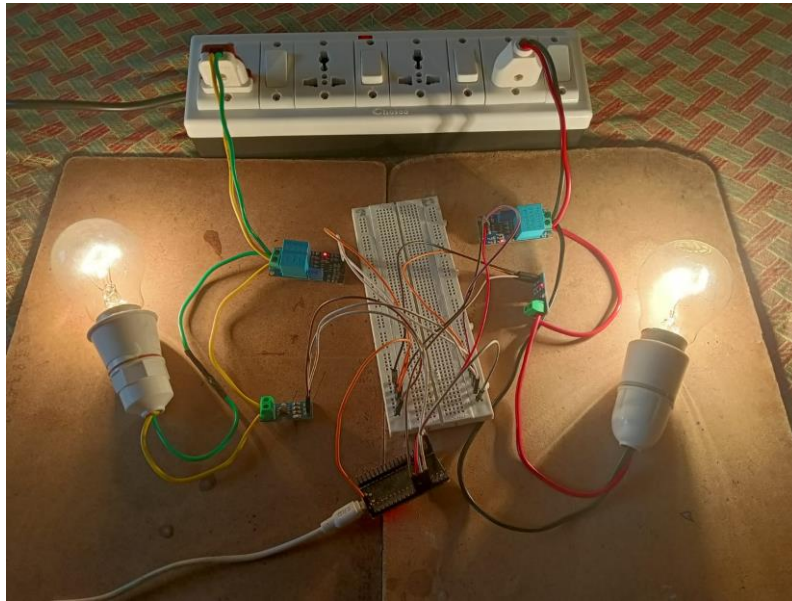


Fig.5.1 Power Consumption Measurement Setup

5.2 POWER BI ANALYSIS

Energy Consumption and Usage Analysis

"A Comprehensive Overview of Energy Usage Metrics for 60W and 100W Bulbs"

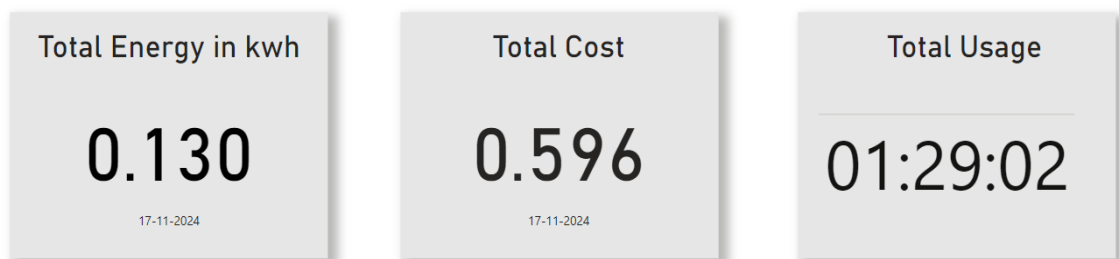


Fig.5.2.1 Consumption Data Overview

Breaking Down Energy Patterns:

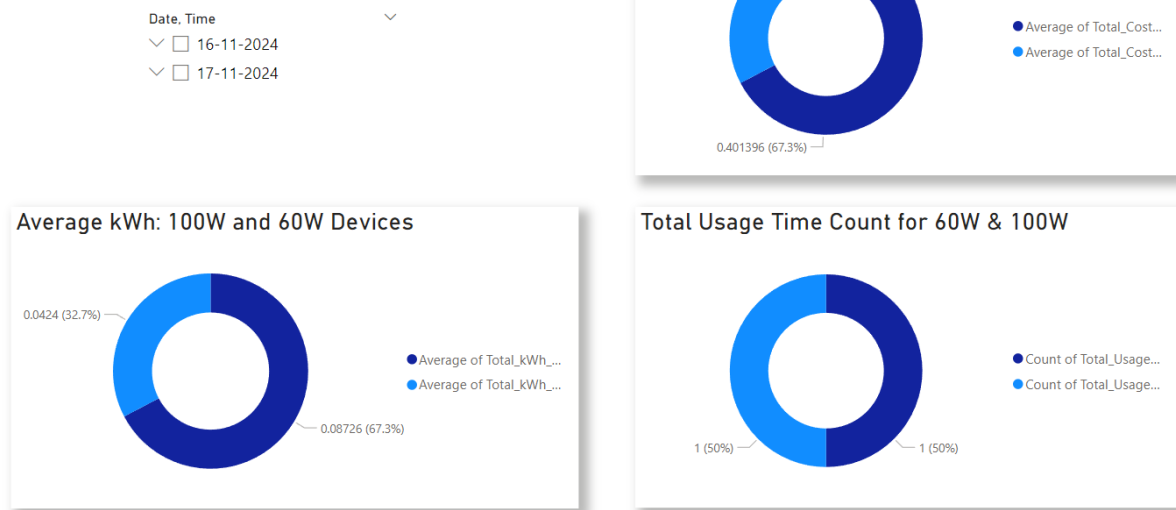


Fig.5.2.2 BREAKING DOWN ENERGY PATTERNS

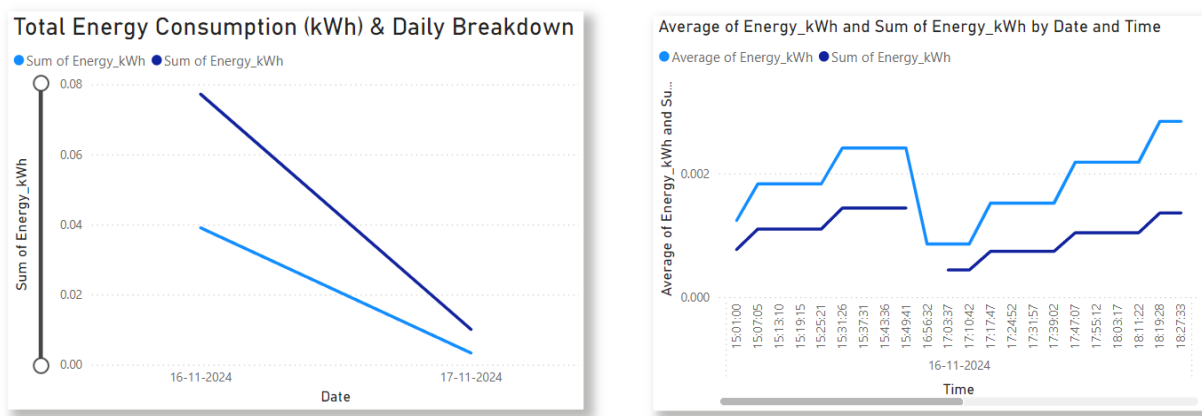


Fig.5.2.3 ENERGY USAGE PATTERS BY DATE AND TIME

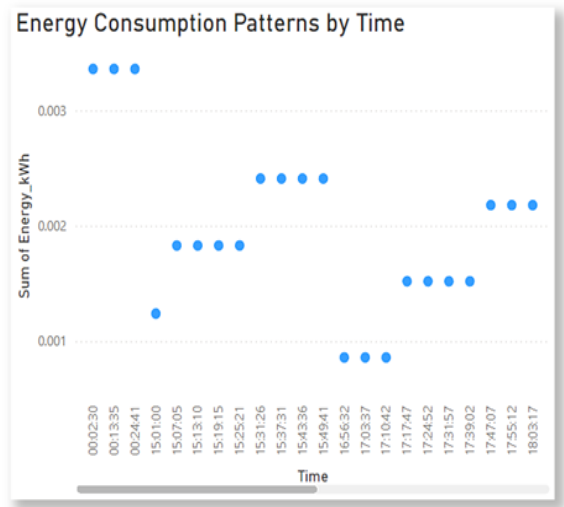
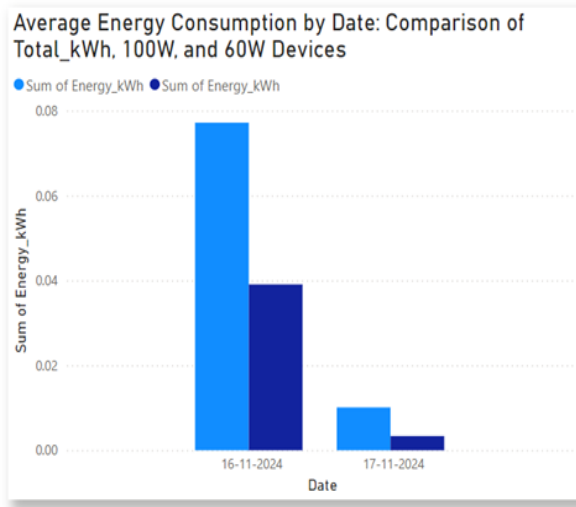


Fig.5.2.4 ENERGY USAGE PATTERS BY DATE AND TIME

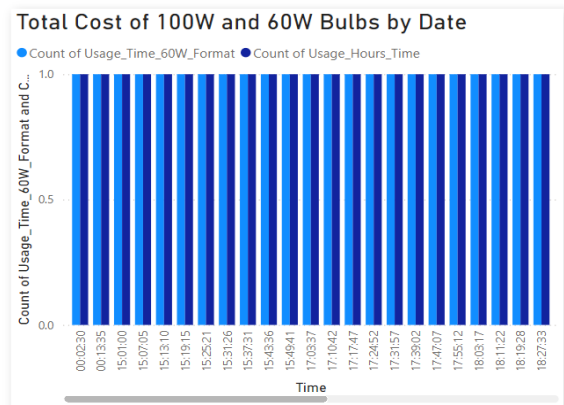
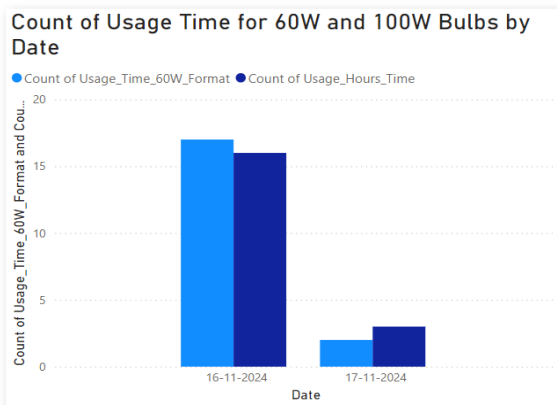


Fig.5.2.5 CHARTS TO REVEAL DEEPER PATTERNS IN USAGE TIME

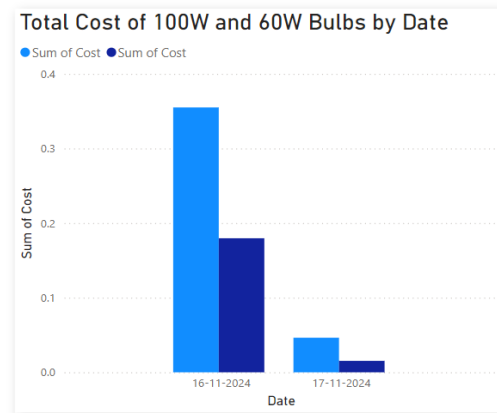
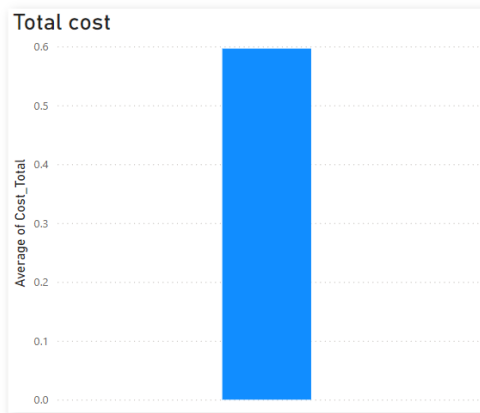


Fig.5.2.6 CHARTS TO REVEAL DEEPER PATTERNS IN COST

6.CONCLUSION

In conclusion, the Energy Monitoring System designed in this project successfully integrates NodeMCU ESP32, current sensors (ACS712), voltage sensors (ZMPT101B), and other essential components to monitor and log the energy consumption of connected appliances. The system uses data streaming to log real-time data, storing the readings in CSV files which are saved in a dedicated folder. This data can then be used for visualization in Power BI for further analysis.

By utilizing NodeMCU ESP32 for seamless connectivity and efficient data transmission, the system enables real-time monitoring and management of energy usage. The integration of current and voltage sensors allows for accurate tracking of power consumption, which can help optimize energy usage and reduce costs.

project offers a practical solution for energy efficiency through IoT technology, making it suitable for both personal and industrial applications. The collected data can be analyzed to identify patterns and areas for energy saving. Future enhancements could include remote control features, cloud integration for real-time monitoring, and predictive analytics for more advanced energy management

7. FUTURE ENHANCEMENT

1. **Cost Reduction:**

The system can help industries optimize energy consumption, reducing electricity costs by identifying areas of waste.

2. **Automated Fault Detection:**

The system can detect abnormal energy usage, alerting administrators to appliance malfunctions and preventing costly repairs.

3. **Production Efficiency:**

Monitoring energy consumption during production will help industries improve efficiency and reduce unnecessary energy use.

4. **Smart Grid Integration:**

The system could connect with smart grids, adjusting energy use based on demand to lower costs and ensure a stable power supply.

5. **Renewable Energy Integration:**

The system can integrate with renewable energy sources like solar power, reducing dependency on traditional energy and promoting sustainability.

6. **Predictive Maintenance:**

By analyzing historical energy data, the system can predict when equipment is likely to fail or require maintenance, reducing downtime and enhancing operational longevity.

REFERENCES:

1. Katekar, V., Deshmukh, S., & Elsheikh, A. (2020). Assessment and Way forward for Bangladesh on SDG-7: Affordable and Clean Energy. *ResearchGate*.
https://www.researchgate.net/publication/344954126_Assessment_and_Way_Forward_for_Bangladesh_on_SDG-7_Affordable_and_Clean_Energy
2. Elorbany, K., Bayılmış, C., & Kaç, S. B. (2021). A smart socket equipped with IoT technologies for energy management of electrical appliances. *Sakarya University Journal of Computer and Information Sciences*, 4(3), 347–353.
<https://doi.org/10.35377/saucis.863272>
3. Durgan, J., Rodríguez-Martínez, M., & Rouse, B. (2023). Green Labs: a guide to developing sustainable science in your organization. *Immunology & Cell Biology*, 101(4), 289–301. <https://doi.org/10.1111/imcb.12624>
4. Sai, H., Nidumolu, V. A., & Panchagnula, V. M. (2022). IoT BASED REAL-TIME ENERGY MONITORING. *ResearchGate*. <https://doi.org/10.24507/icicel.16.08.827>
5. J. S. Bassi, E. G. Dada, and M. S. Abdullahi, “Development of internet of things (IoT) based energy consumption monitoring and device control system,” *Nipes Journal of Science and Technology Research*, vol. 2, no. 3, p. 85, Aug. 2020.
<https://doi.org/10.37933/nipes/2.3.2020.9>
6. L. W. Zhang, K. Shanmugam, and M. E. Rana, “IoT based intelligent energy monitoring system for computer laboratories,” *2022 International Conference on Decision Aid Sciences and Applications (DASA)*, Mar. 2022.
<https://doi.org/10.1109/dasa54658.2022.9765295>

7. Salisu, A., Bugaje, A., & Loko, A. Z. (2021, June 12). IoT BASED HOUSEHOLD ELECTRICITY ENERGY MONITORING AND CONTROL. *FUDMA Journal of Sciences*, 4(4), 77–84. <https://doi.org/10.33003/fjs-2020-0404-466>

8. Sowjanya, M. B. T., Tadiparth, H., Pidiyar, K. V., Gaddala, S. S., & Sabavath, D. (2023, December 2). IoT- Based Smart Energy Meter. *International Journal of Research Publication and Reviews*, 4(12), 3175–3183. <https://doi.org/10.55248/gengpi.4.1223.123314>

9. Sanya, W., Bajpai, G., Kombo, O., & Twahirwa, E. (2022, July 16). Real-Time Data Analytics for Monitoring Electricity Consumption Using IoT Technology. *Tanzania Journal of Engineering and Technology*, 41(1), 28–37. <https://doi.org/10.52339/tjet.vi.770>

10. P. Singh, S. Shokeen and S. Garg, "Generation of power system using renewable resources for sustainable development leading upcoming technologies: an analysis," *2022 OPJU International Technology Conference on Emerging Technologies for Sustainable Development (OTCON)*, Raigarh, Chhattisgarh, India, 2023, pp. 1-5. <https://doi.org/10.1109/OTCON56053.2023.10114001>